

Technical Solution Document

Batch Data Processing for Rental Marketplace Analytics

1. Introduction

1.1 Purpose

The purpose of this document is to outline the technical architecture and implementation details for a batch data processing pipeline. The goal is to enable analytical reporting on rental listings and user interactions for a rental marketplace platform.

1.2 Scope

This project involves:

- Extracting data from AWS Aurora MySQL and storing it in Amazon S3.
- Loading the data into Amazon Redshift for further processing.
- Implementing a multi-layer data architecture in Redshift (Raw, Curated, Presentation).
- Using AWS Glue for ETL operations.
- Orchestrating the workflow using AWS Step Functions.

1.3 Definitions, Acronyms, and Abbreviations

- ETL: Extract, Transform, Load
- AWS: Amazon Web Services
- S3: Amazon Simple Storage Service
- Redshift: Amazon Redshift Data Warehouse
- Step Functions: AWS Step Functions for orchestration
- Glue: AWS Glue for data transformation

1.4 References

- AWS Glue Documentation: <https://docs.aws.amazon.com/glue/latest/dg/>
- AWS Step Functions Documentation: <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>

1.5 Overview

This document describes the architecture, components, and implementation details of the batch ETL pipeline. It also includes system constraints, use cases, and deployment strategies.

2. Architectural Representation

2.1 Architecture Overview

The architecture follows a batch ETL pipeline model and consists of:

- **Data Extraction:** Extracting data from AWS Aurora MySQL.
 - **Storage Layer:** Storing raw data in S3.
 - **Data Loading:** Copying raw data to Redshift.
 - **Transformation Layer:** Processing raw data into a curated format.
 - **Presentation Layer:** Preparing data for analytical queries.
 - **Orchestration:** Managing workflow execution using AWS Step Functions.
-

3. Architectural Goals and Constraints

3.1 System-wide Design Decisions

- Batch processing over real-time processing due to cost efficiency.
- Serverless architecture using AWS Glue for cost optimization.
- Scalable storage with Amazon S3 and Redshift for large datasets.

3.2 Architectural Constraints

- AWS Aurora MySQL as the primary data source.
 - AWS services such as Glue, S3, and Redshift must be used.
 - Data must be transformed in compliance with analytical needs.
-

4. Use-Case View

4.1 Use-Case Model

- **Extract data from AWS Aurora MySQL.**
- **Store extracted data in Amazon S3.**
- **Load data into Amazon Redshift (Raw layer).**
- **Transform raw data using AWS Glue.**
- **Load transformed data into Redshift (Curated layer).**
- **Compute business metrics using SQL queries.**

4.2 Use-Case Realizations

Each ETL step is executed as an AWS Glue job within an AWS Step Functions workflow.

5. Logical View

5.1 Major Logical Components

- **AWS Aurora MySQL: Source database.**
 - **Amazon S3: Storage for raw and transformed data.**
 - **AWS Glue: ETL processing engine.**
 - **Amazon Redshift: Data warehouse.**
 - **AWS Step Functions: Workflow orchestration.**
-

6. Process View

6.1 Concurrent Processes and Synchronization

- **Step Functions ensure sequential execution of ETL jobs.**
 - **AWS Glue jobs run in parallel where applicable.**
-

7. Deployment View

7.1 Mapping Software Components to Hardware

- **AWS Glue Jobs: Serverless compute environment.**
- **Amazon S3: Storage layer.**

- **Amazon Redshift Cluster: Compute and storage for analytical queries.**
-

8. Implementation View

8.1 Development Environment

- **AWS Glue: Python-based ETL scripts.**
- **AWS Lambda (if needed for auxiliary tasks).**
- **SQL for Redshift queries.**

8.2 Configuration Management

- **AWS IAM Roles for Glue, Redshift, and Step Functions.**
 - **AWS CloudFormation for infrastructure automation.**
-

9. Data View

9.1 Data Storage and Retrieval Mechanisms

- **Raw data: Stored in Amazon S3.**
- **Structured data: Managed in Amazon Redshift.**

9.2 Data Integrity and Security

- **S3 Bucket Policies to restrict access.**
 - **Redshift IAM Authentication for secure data access.**
-

10. Size and Performance

10.1 Performance Benchmarks

- **Redshift queries should return within 2-5 seconds for analytical queries.**
- **AWS Glue processing should complete within 30 minutes for batch jobs.**

10.2 Scalability Constraints

- **Redshift concurrency scaling for handling multiple queries.**
- **S3 storage automatically scales as data grows.**

11. Quality Attributes

11.1 Usability

- Queries should be optimized for fast reporting.
- Data transformation should maintain high accuracy.

11.2 Reliability

- Step Functions ensure retries and error handling.

11.3 Maintainability

- Code should be modular and follow best practices for ETL pipelines.

11.4 Security

- Use AWS KMS for data encryption.
 - Implement IAM roles for least privilege access.
-

12. Appendices

12.1 Glossary

- ETL: Extract, Transform, Load
- AWS Glue: Managed ETL service
- Step Functions: AWS service for orchestrating workflows

12.2 Index

- Batch Processing → Section 2
- Data Storage → Section 9

12.3 Revision History

Version	Date	Author	Changes
1.0	2025-03-28	Zuki	Initial Draft